06. 전체 시스템 통합 및 동작 원리

6.1 시스템 아키텍처 전체 개요

6.1.1 계층 구조

```
┌─────────────────────────────────────────────────────────┐
│ Application Layer │
│ (테스트 코드, 사용자) │
└─────────────────────────────────────────────────────────┘
↓
┌─────────────────────────────────────────────────────────┐
│ VM Layer (vm.rs) │
│ - 프로그램 실행 │
│ - 스택 관리 │
│ - 추적 생성 │
└─────────────────────────────────────────────────────────┘
↓
┌─────────────────────────────────────────────────────────┐
│ AIR Layer (p3.rs) │
│ - 제약 조건 정의 │
│ - 추적 변환 │
```

```
└──────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────┐
| Plonky3 STARK Layer |
| - 다항식 커밋먼트 |
| - FRI 프로토콜 |
| - 증명 생성/검증 |
└──────────────────────────────────────────────────────────┘
```

6.2 데이터 흐름 분석

6.2.1 전체 데이터 흐름

```
Program (Vec<Instructions>)
↓
VM::new(program)
↓
VM::run()
↓
Vec<VMState> (내부 추적)
↓
VM::get_trace()
↓
```

```
Vec<[Mersenne31; 11]> (11컬럼 추적)
↓
p3::get_trace()
↓
RowMajorMatrix<F> (행렬 형태)
↓
prove()
↓
Proof (STARK 증명)
↓
verify()
↓
Accept/Reject
```

6.2.2 각 단계별 데이터 변환

단계 1: 프로그램 정의

```rust
let  program = vec![
Instructions::Push(Mersenne31::from_canonical_u32(10)),
Instructions::Push(Mersenne31::from_canonical_u32(20)),
Instructions::Add,
];
```

데이터 형태:

```
Vec<Instructions> = [Push(10), Push(20), Add]
```

단계 2: VM 실행

```
let mut vm = VM::new(program);
vm.run()?;
```

내부 추적 생성:

```
Vec<VMState> = [
VMState { stack: [10,0,0,0], instruction: Push(10), extra_data: 0 },
VMState { stack: [20,10,0,0], instruction: Push(20), extra_data: 0 },
VMState { stack: [30,0,0,0], instruction: Add, extra_data: 0 },
]
```

단계 3: 추적 변환

```
let trace = vm.get_trace();
```

11컬럼 추적:

```
Vec<[Mersenne31; 11]> = [
[0,0,0,0,0,0,0,0,0,0,0], // 초기 상태
[10,0,0,0,10,1,0,0,0,0,0], // Push(10)
```

```
[20,10,0,0,20,1,0,0,0,0,0], // Push(20)
[30,0,0,0,0,0,1,0,0,0,0], // Add
[30,0,0,0,0,0,0,0,0,0,0], // 패딩
[30,0,0,0,0,0,0,0,0,0,0], // 패딩
[30,0,0,0,0,0,0,0,0,0,0], // 패딩
[30,0,0,0,0,0,0,0,0,0,0], // 패딩
]
```

단계 4: 행렬 변환

```
let trace_matrix = get_trace(vm);
```

RowMajorMatrix:

```
RowMajorMatrix {
values: [0,0,0,0,0,0,0,0,0,0,0, 10,0,0,0,10,1,0,0,0,0,0, ...],
width: 11,
height: 8
}
```

단계 5: 증명 생성

```
let proof = prove(&config, &vmair, &mut challenger, trace_matrix, &vec![]);
```

증명 구조:

```
Proof {
trace_commitments: [MerkleRoot₁ ],
quotient_commitments: [MerkleRoot₂ ],
fri_proof: FriProof { ... },
query_openings: [Opening₁ , Opening₂ , ...],
pow_nonce: 12345,
}
```

6.3 완전한 실행 예시

6.3.1 복잡한 프로그램

```
let  program = vec![
Instructions::Push(Mersenne31::from_canonical_u32(10)), // [10,0,0,0]
Instructions::Push(Mersenne31::from_canonical_u32(20)), // [20,10,0,0]
Instructions::Add, // [30,0,0,0]
Instructions::Push(Mersenne31::from_canonical_u32(40)), // [40,30,0,0]
Instructions::Sub, // [10,0,0,0]
Instructions::Push(Mersenne31::from_canonical_u32(2)), // [2,10,0,0]
Instructions::Mul, // [20,0,0,0]
Instructions::Push(Mersenne31::from_canonical_u32(23)), // [23,20,0,0]
Instructions::Div, // [1,0,0,0] (나머지 3)
```

```
];
```

6.3.2 단계별 스택 상태

| 단계 | 명령어 | 실행 전 스택 | 실행 후 스택 | 설명 |

|------|--------|----------|----------|------|

| 0 | - | [0,0,0,0] | [0,0,0,0] | 초기 상태 |

| 1 | Push(10) | [0,0,0,0] | [10,0,0,0] | 10 푸시 |

| 2 | Push(20) | [10,0,0,0] | [20,10,0,0] | 20 푸시 |

| 3 | Add | [20,10,0,0] | [30,0,0,0] | 20+10=30 |

| 4 | Push(40) | [30,0,0,0] | [40,30,0,0] | 40 푸시 |

| 5 | Sub | [40,30,0,0] | [10,0,0,0] | 40-30=10 |

| 6 | Push(2) | [10,0,0,0] | [2,10,0,0] | 2 푸시 |

| 7 | Mul | [2,10,0,0] | [20,0,0,0] | 2×10=20 |

| 8 | Push(23) | [20,0,0,0] | [23,20,0,0] | 23 푸시 |

| 9 | Div | [23,20,0,0] | [1,0,0,0] | 23÷20=1 나머지 3 |

6.3.3 생성되는 추적 테이블

```
Row | s[0] | s[1] | s[2] | s[3] | pval | isPush | isAdd | isSub | isMul | isDiv

----|------|------|------|------|------|--------|-------|-------|-------|-------

0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0

1 | 10 | 0 | 0 | 0 | 10 | 1 | 0 | 0 | 0 | 0 | 0

2 | 20 | 10 | 0 | 0 | 20 | 1 | 0 | 0 | 0 | 0 | 0

3 | 30 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0
```

```
4 | 40 | 30 | 0 | 0 | 40 | 1 | 0 | 0 | 0 | 0 | 0

5 | 10 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0

6 | 2 | 10 | 0 | 0 | 2 | 1 | 0 | 0 | 0 | 0 | 0

7 | 20 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0

8 | 23 | 20 | 0 | 0 | 23 | 1 | 0 | 0 | 0 | 0 | 0

9 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 3

10 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0

... (패딩 행들)

15 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0
```

6.3.4 제약 조건 검증 예시

Row 2 → Row 3 (Add 연산):

```
local = [20, 10, 0, 0, 20, 1, 0, 0, 0, 0, 0]

next = [30, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0]


제약 1: next[6] * (next[0] - local[0] - local[1])

= 1 * (30 - 20 - 10)

= 1 * 0

= 0 ✓


제약 2: next[6] * (next[1] - local[2])

= 1 * (0 - 0)
```

```
= 0 ✓
```

Row 8 → Row 9 (Div 연산):

```
local = [23, 20, 0, 0, 23, 1, 0, 0, 0, 0, 0]

next = [1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 3]


제약: next[9] * (local[1] * next[0] + next[10] - local[0])

= 1 * (20 * 1 + 3 - 23)

= 1 * (20 + 3 - 23)

= 1 * 0

= 0 ✓


검증: 23 = 20 × 1 + 3 (올바른 나눗셈)
```

6.4 모듈 간 상호작용

6.4.1 vm.rs와 p3.rs의 상호작용

```rust
// vm.rs에서

impl VM {

pub  fn  get_trace(&self) -> Vec<[Mersenne31; 11]> {

// VM 내부 추적을 11컬럼 형태로 변환

}
```

```
}


// p3.rs에서
pub fn get_trace<F: Field>(vm: VM) -> RowMajorMatrix<F> {
let trace = vm.get_trace(); // VM의 추적 가져오기
// 필드 변환 및 패딩
// RowMajorMatrix 생성
}
```

데이터 흐름:

```
VM (vm.rs)
↓ get_trace()
Vec<[Mersenne31; 11]>
↓ p3::get_trace()
RowMajorMatrix<F>
↓ prove()
Proof
```

6.4.2 AIR와 STARK의 상호작용

```
// AIR 정의 (p3.rs)
impl<AB: AirBuilder> Air<AB> for VMAir {
fn eval(&self, builder: &mut AB) {
```

```
// 제약 조건 정의
}
}

// STARK 증명 생성
let proof = prove(
&config,
&vmair, // AIR 전달
&mut challenger,
trace,
&vec![]
);
```

상호작용:

```
VMAir::eval()
↓ 제약 조건 제공
prove()
↓ 제약 조건 평가
Constraint Polynomials
↓ 커밋먼트
Proof
```

6.5 타입 시스템과 제네릭

6.5.1 필드 타입의 흐름

```
// VM에서는 구체적 타입 사용
VM {
stack: [Mersenne31; 4],
// ...
}

// AIR에서는 제네릭 타입
impl<F: Field> BaseAir<F> for VMAir {
// F는 임의의 필드
}

// 증명 생성에서 구체화
type Val = Mersenne31;
let  trace = get_trace::<Val>(vm);
```

타입 변환:

```
Mersenne31 (VM)
↓ as_canonical_u32()
u32
```

```
↓ F::from_canonical_u32()

F (제네릭 필드)
```

6.5.2 제네릭의 장점

1. **유연성**: 다른 필드로 쉽게 전환 가능
2. **재사용성**: 같은 AIR을 다른 필드에서 사용
3. **타입 안전성**: 컴파일 타임에 타입 체크

6.6 에러 처리

6.6.1 VM 실행 에러

```rust
pub fn run(&mut self) -> Result<(), String> {
// 실행 로직
Ok(())
}
```

가능한 에러:

- 스택 언더플로우 (현재 구현에서는 체크 안 함)
- 0으로 나누기 (Mersenne31 필드에서는 역원 존재)
- 잘못된 명령어 (타입 시스템으로 방지)

6.6.2 증명 생성 에러

```rust
let proof = prove(&config, &vmair, &mut challenger, trace, &vec![]);
```

```
// 내부적으로 panic 또는 에러 반환
```

가능한 에러:

- 제약 조건 위반
- FRI 프로토콜 실패
- 메모리 부족

6.7 성능 특성

6.7.1 시간 복잡도

| 작업 | 복잡도 | 설명 |

|------|--------|------|

| VM 실행 | O(n) | n = 명령어 수 |

| 추적 생성 | O(n) | 각 명령어마다 상태 기록 |

| 패딩 | O(m) | m = 패딩 행 수 |

| 증명 생성 | O(n log n) | FFT 기반 |

| 증명 검증 | O(log n) | 쿼리 기반 |

6.7.2 공간 복잡도

| 데이터 | 크기 | 설명 |

|--------|------|------|

| VM 상태 | O(1) | 고정 크기 스택 |

| 내부 추적 | O(n) | n = 명령어 수 |

| 최종 추적 | O(m) | m = 2^k (패딩 후) |

| 증명 | O(log n) | 간결한 증명 |

6.7.3 실제 성능 예시

```
프로그램 크기: 9개 명령어

패딩 후 크기: 16행 (2^4)

증명 생성 시간: ~100ms

증명 크기: ~10KB

검증 시간: ~10ms
```

6.8 확장 가능성

6.8.1 새로운 명령어 추가

```
// 1. Instructions 열거형에 추가

pub  enum Instructions {

// 기존 명령어들...

Mod, // 새로운 명령어

}


// 2. VM::run에 구현 추가

Instructions::Mod => {

self.perform_operation(|a, b| a % b, None::<BinaryOp>)?;

}
```

```
// 3. get_trace에 인코딩 추가
Instructions::Mod => {
final_trace.push([
i.stack[0], i.stack[1], i.stack[2], i.stack[3],
Mersenne31::ZERO,
Mersenne31::ZERO, Mersenne31::ZERO, Mersenne31::ZERO,
Mersenne31::ZERO, Mersenne31::ZERO,
Mersenne31::ONE, // is_mod 플래그
i.extra_data,
]);
}

// 4. AIR에 제약 조건 추가
builder
.when_transition()
.assert_zero(next[10] * (local[0] - local[1] * next[0] - next[11]));
```

6.8.2 더 큰 스택

```
// 스택 크기 변경
pub struct VM {
stack: [Mersenne31; 8], // 4 → 8
```

```
// ...
}

// 추적 컬럼 수 조정
impl<F: Field> BaseAir<F> for VMAir {
fn width(&self) -> usize {
15 // 11 → 15 (스택 8개 + 나머지)
}
}
```

6.8.3 메모리 추가

```
pub struct VM {
stack: [Mersenne31; 4],
memory: HashMap<u32, Mersenne31>, // 새로운 메모리
// ...
}

// 새로운 명령어
pub enum Instructions {
// ...
Load(u32), // 메모리에서 로드
Store(u32), // 메모리에 저장
```

```
}
```

6.9 디버깅과 테스트

6.9.1 추적 출력

```
pub fn get_trace<F: Field>(vm: VM) -> RowMajorMatrix<F> {
let trace = vm.get_trace();
println!("{:?}", trace); // 추적 출력
// ...
}
```

출력 예시:

```
[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[10, 0, 0, 0, 10, 1, 0, 0, 0, 0, 0],
[20, 10, 0, 0, 20, 1, 0, 0, 0, 0, 0],
[30, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0]]
```

6.9.2 단위 테스트

```
#[test]
fn check_add_operation() {
let program = vec![
```

```
Instructions::Push(Mersenne31::from_canonical_u32(10)),
Instructions::Push(Mersenne31::from_canonical_u32(20)),
Instructions::Add,
];

let mut vm = VM::new(program);
vm.run().unwrap();

assert_eq!(
vm.stack[0],
Mersenne31::from_canonical_u32(30)
);
}
```

6.9.3 통합 테스트

```
#[test]
fn test_end_to_end() {
let program = vec![/* ... */];
let mut vm = VM::new(program);
vm.run().unwrap();
let vmair = VMAir {};
vmair.generate_proof(vm); // 증명 생성 및 검증
```

```
// 에러 없으면 성공
}
```

6.10 실제 사용 시나리오

6.10.1 간단한 계산 검증

```
// 계산: (10 + 20) * 2 = 60
let program = vec![
Instructions::Push(Mersenne31::from_canonical_u32(10)),
Instructions::Push(Mersenne31::from_canonical_u32(20)),
Instructions::Add,
Instructions::Push(Mersenne31::from_canonical_u32(2)),
Instructions::Mul,
];

let mut vm = VM::new(program);
vm.run()?;

// 결과 확인
assert_eq!(vm.stack[0], Mersenne31::from_canonical_u32(60));

// 증명 생성
```

```
let  vmair = VMAir {};

vmair.generate_proof(vm);
```

6.10.2 복잡한 수식 검증

```
// 계산: ((a + b) * c - d) / e

// a=5, b=3, c=4, d=2, e=3

// ((5+3)*4-2)/3 = (8*4-2)/3 = (32-2)/3 = 30/3 = 10


let  program = vec![

Instructions::Push(Mersenne31::from_canonical_u32(5)), // a

Instructions::Push(Mersenne31::from_canonical_u32(3)), // b

Instructions::Add, // a+b

Instructions::Push(Mersenne31::from_canonical_u32(4)), // c

Instructions::Mul, // (a+b)*c

Instructions::Push(Mersenne31::from_canonical_u32(2)), // d

Instructions::Sub, // (a+b)*c-d

Instructions::Push(Mersenne31::from_canonical_u32(3)), // e

Instructions::Div, // ((a+b)*c-d)/e

];
```

6.11 시스템의 한계와 개선 방향

6.11.1 현재 한계

1. **고정 스택 크기**: 4개 요소로 제한
2. **메모리 없음**: 복잡한 프로그램 실행 불가
3. **제어 흐름 없음**: 조건문, 반복문 없음
4. **영지식성 없음**: 추적이 완전히 노출됨

6.11.2 개선 방향

1. **동적 메모리**: RAM 추가
2. **제어 흐름**: Jump, Branch 명령어
3. **영지식성**: 추적 무작위화
4. **최적화**: 더 효율적인 제약 조건

6.12 요약

이 plonky3 zkVM 구현은 다음을 보여줍니다:

1. **완전한 zkVM 파이프라인**: 실행 → 추적 → 증명 → 검증
2. **모듈화된 설계**: VM, AIR, STARK 분리
3. **실용적인 구현**: 실제 동작하는 코드
4. **교육적 가치**: zkVM의 핵심 개념 학습

이 시스템은 간단하지만 zkVM의 모든 핵심 요소를 포함하고 있어, 더 복잡한 zkVM을 이해하는 데 훌륭한 출발점이 됩니다.